STUDENT PERFORMANCE CLASSIFICATION

A Machine Learning Mini Project using K-Nearest Neighbors (KNN)

Student Name	D. Charan Tej
Roll Number	24501A4411
Branch	CSD
Dataset	Students Performance Dataset
Methodology	CRISP-DM
Model	K-Nearest Neighbors (KNN)

Project objective: classify a student's GradeClass using academic, demographic, and activity-related attributes while excluding direct identifiers and GPA to reduce target leakage.

Dataset file supplied by the student; all reported metrics and figures in this report were generated by executing the workflow on that file.

1. Business Understanding

1.1 Problem Statement and Relevance

Educational performance is influenced by multiple factors, including study time, absences, access to tutoring, parental support, and participation in extracurricular activities. This project builds a supervised classification model that predicts the dataset's GradeClass label from the available non-leakage features.

The task is a five-class classification problem. The intention is to demonstrate an end-to-end machine-learning workflow—not to make high-stakes decisions about individual students.

1.2 Expected Benefits

A KNN model provides a straightforward baseline for learning how similarity between student records can be used for classification. The workflow demonstrates preprocessing, stratified splitting, hyperparameter tuning, and evaluation using metrics that reflect performance across classes.

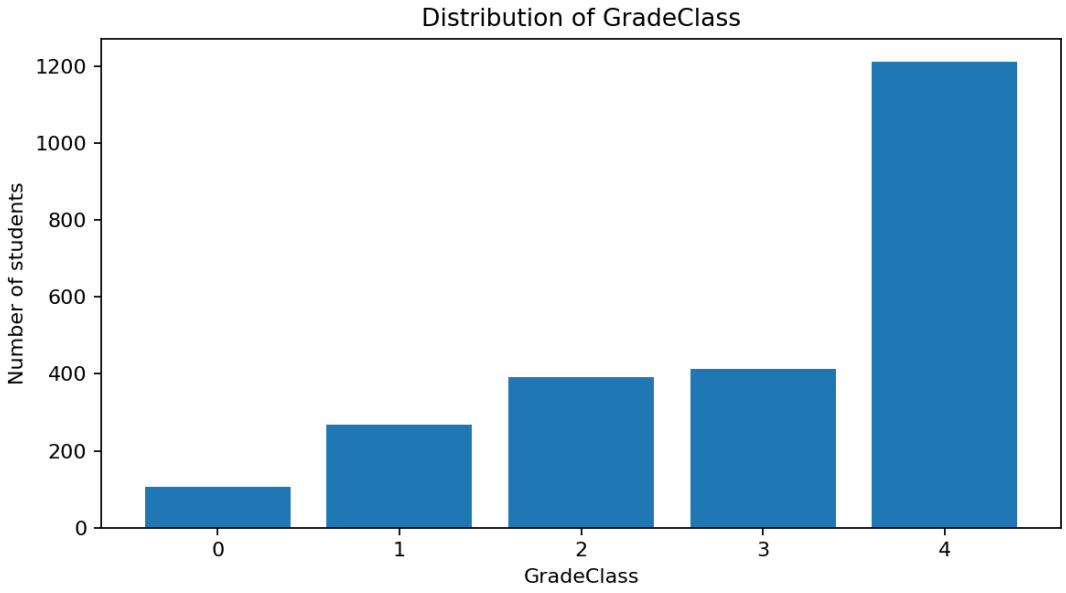
2. Data Understanding

2.1 Dataset Structure

The supplied CSV contains **2,392 rows** and **15 columns**. The target is **GradeClass**, encoded as classes 0 through 4. The class counts and feature descriptions below were computed from the uploaded file.

Item	Observed value
Rows	2,392
Columns	15
Target	GradeClass
Target classes	0, 1, 2, 3, 4
Missing cells	0
Duplicate rows	0

2.2 Target distribution



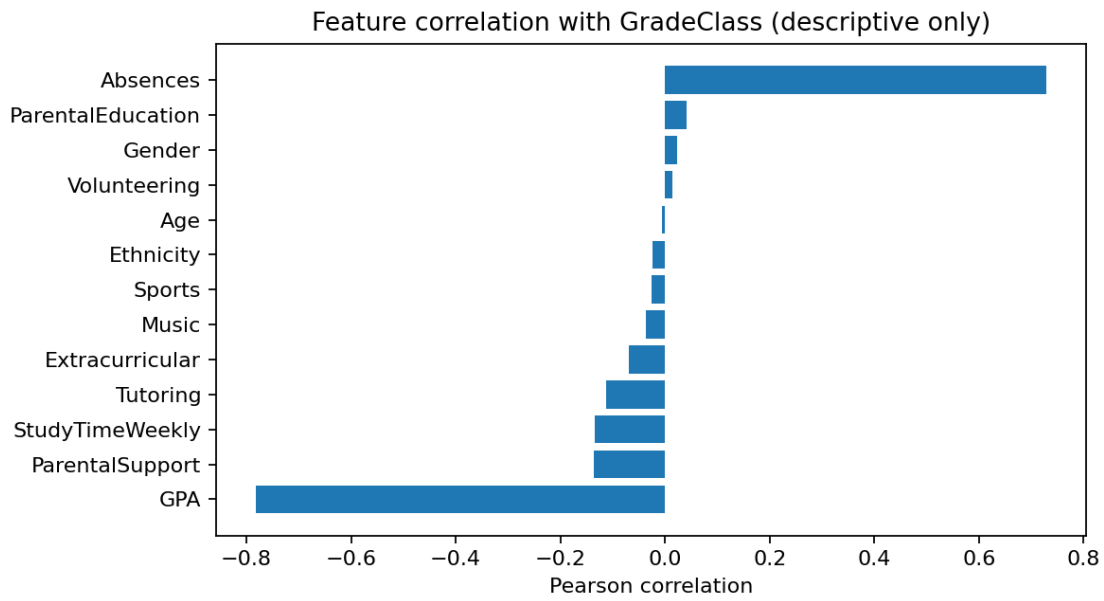
2.3 Data Dictionary

Column	Description / role
StudentID	Record identifier; excluded from model
Age	Student age
Gender	Encoded gender category
Ethnicity	Encoded ethnicity category
ParentalEducation	Encoded parental education category
StudyTimeWeekly	Weekly study time in hours
Absences	Number of absences
Tutoring	Binary tutoring indicator
ParentalSupport	Encoded parental support level
Extracurricular	Binary extracurricular participation
Sports	Binary sports participation
Music	Binary music participation
Volunteering	Binary volunteering participation
GPA	Grade point average; excluded due to target leakage risk
GradeClass	Target label, classes 0–4

2.4 Data Quality Observations

The uploaded file has 0 missing cells and 0 duplicate rows under pandas' exact-row duplicate check. All listed columns are numeric in this file, including categorical variables represented as integer codes. The baseline pipeline imputes missing numeric values with the median and standardizes features.

Feature correlation with target (descriptive only)



Correlation is not evidence of causation. Since GPA is closely related to the GradeClass label, GPA is excluded from the predictive feature set to avoid giving the model a target-derived shortcut.

3. Data Preparation

3.1 Feature and target selection

The target variable is GradeClass. StudentID is removed because it is an identifier rather than a meaningful student characteristic. GPA is also removed because the target class is derived from academic performance and GPA could create leakage.

3.2 Train-test split

The dataset is split into 80% training and 20% testing data using stratification, random_state=42. Stratification preserves the class proportions in both subsets. The split produced **1,913 training rows** and **479 test rows**.

3.3 Preprocessing

A scikit-learn Pipeline applies median imputation and StandardScaler before KNN. Scaling is important because KNN relies on distance: without scaling, features with larger numeric ranges can dominate distance calculations. All preprocessing is fitted within cross-validation/training folds to reduce data leakage.

4. Modelling

4.1 K-Nearest Neighbors

KNN predicts the class of a new record by finding its k nearest training examples in feature space. The majority class among those neighbors is used for prediction when uniform weighting is selected. Manhattan distance (p=1) and Euclidean distance (p=2) were considered.

4.2 Hyperparameter tuning

GridSearchCV searched k values of 3, 5, 7, 9, 11, 13, 15, 17, 19, and 21; neighbor weighting options uniform and distance; and Minkowski distance parameter p=1 or p=2. Five-fold stratified cross-validation on the training set optimized macro F1, which gives each class equal weight.

Setting	Selected / used value
Model	KNeighborsClassifier
Best k	5
Weights	uniform
Distance p	1
CV macro F1	0.4016

5. Evaluation

5.1 Test-set metrics

The final model was evaluated once on the held-out test set. The results below are actual outputs from the uploaded CSV and the specified random seed.

Metric	Test result
Accuracy	0.6075 (60.75%)
Macro precision	0.4501
Macro recall	0.4428
Macro F1-score	0.4420
Test records	479

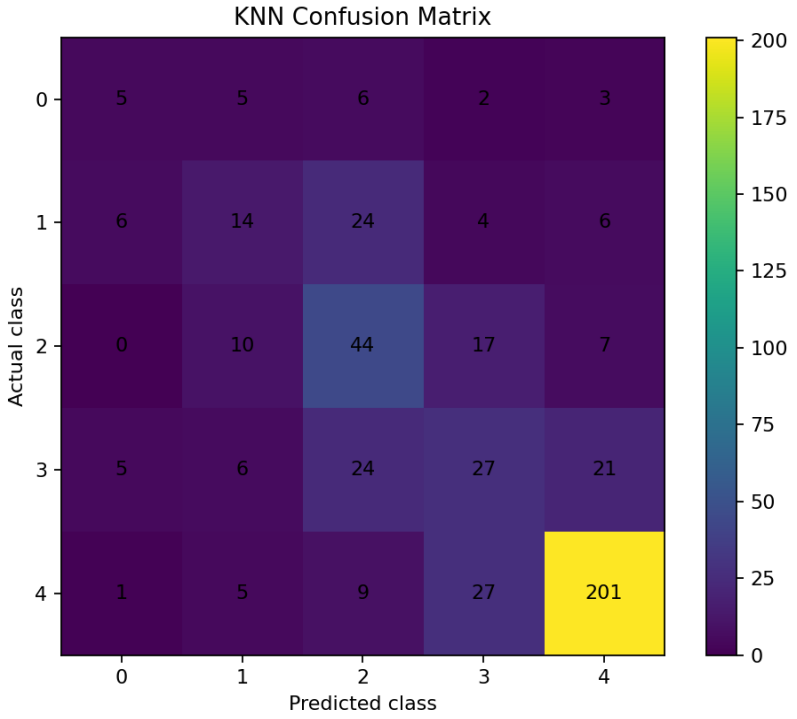
5.2 Classification report

```
              precision    recall  f1-score   support

           0      0.294     0.238     0.263        21
           1      0.350     0.259     0.298        54
           2      0.411     0.564     0.476        78
           3      0.351     0.325     0.338        83
           4      0.845     0.827     0.836       243

    accuracy                          0.608       479
   macro avg      0.450     0.443     0.442       479
weighted avg      0.609     0.608     0.605       479
```

5.3 Confusion matrix



Interpretation: class 4 has the strongest performance and is the most common class in the test set. The model confuses several neighboring middle classes, particularly classes 2 and 3. The overall accuracy should therefore be read alongside macro F1 and class-level recall, rather than in isolation.

6. Deployment and Final Project Summary

6.1 Intended use

This project is a learning prototype. A practical deployment would require a validated interface, documentation of the model's limitations, monitoring for changes in student populations, and review by education professionals. Predictions should not be used as the sole basis for grading, placement, discipline, or other consequential decisions.

6.2 Reproducibility

The results can be reproduced by running the Python workflow in the appendix with the supplied CSV and the same library versions, split seed, search grid, and cross-validation settings. Small differences can occur across software versions.

6.3 Final summary

The KNN model selected k=5, weights='uniform', and p=1. It achieved 60.75% test accuracy and 0.442 macro F1. These results show that KNN captures some structure in the dataset, but performance is uneven across GradeClass labels.

7. Conclusion

The project completed a CRISP-DM-style workflow using the Students Performance Dataset: understanding the data, preparing features, tuning a KNN classifier, and evaluating it on held-out records. The most important methodological choice was excluding StudentID and GPA from model inputs. The class imbalance and confusion among middle classes limit the model's reliability.

Limitations and future work

The data are observational and the encoded categories may hide meaningful differences between groups. Future work could inspect fairness and subgroup performance, test calibration, and gather domain feedback. Any additional model comparisons should be a separate experiment; this report intentionally uses KNN only.

8. Appendix — Executed Python workflow

The core code below describes the preprocessing, split, cross-validation search, and evaluation used to generate the reported results.

Appendix A — Training and tuning code

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("Student_performance_data.csv")
X = df.drop(columns=["StudentID", "GPA", "GradeClass"])
y = df["GradeClass"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 11, 13, 15, 17, 19, 21],
    "knn__weights": ["uniform", "distance"],
    "knn__p": [1, 2]
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(
    pipeline, param_grid, scoring="f1_macro", cv=cv, n_jobs=-1
)
search.fit(X_train, y_train)
pred = search.predict(X_test)
```

Appendix B — Evaluation code and observed outputs

```
print("Best parameters:", search.best_params_)
print("Best CV macro F1:", round(search.best_score_, 4))
print("Test accuracy:", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred, zero_division=0))
print("Confusion matrix:")
print(confusion_matrix(y_test, pred))

# Actual run output (rounded):
# Best parameters: {'knn__n_neighbors': 5,
#                   'knn__p': 1, 'knn__weights': 'uniform'}
# Best CV macro F1: 0.4016
# Test accuracy: 0.6075
# Macro precision: 0.4501
# Macro recall: 0.4428
# Macro F1-score: 0.4420
```

Actual confusion matrix (rows = actual class; columns = predicted class)

Actual \ Predicted	0	1	2	3	4
0	5	5	6	2	3
1	6	14	24	4	6
2	0	10	44	17	7
3	5	6	24	27	21
4	1	5	9	27	201

Dataset source: Students Performance Dataset by Rabie El Kharoua, supplied as a CSV by the student. The report's counts, metrics, confusion matrix, and plots were generated from the uploaded file.